High-Level Design (HLD) — SOC Training Simulator

Field	Value
Version	1.1
Date	February 26, 2026
Author	Abdullah Al-Hussein
Status	Released (updated post-security hardening)

---

1. Executive Summary

The **SOC Training Simulator** is an open-source, AI-powered platform that prepares Security Operations Center (SOC) analysts for real-world incidents before they face one. It provides a realistic investigation workspace where trainees analyze simulated logs, collect evidence, build timelines, and answer checkpoint questions — all while receiving Socratic guidance from an AI-powered SOC Mentor.

The platform serves three user roles — **Admins** (system management), **Trainers** (session orchestration and monitoring), and **Trainees** (investigation and learning) — and is designed for cybersecurity bootcamps, university programs, enterprise SOC teams, and self-study analysts.

Key Differentiators:

- AI at three levels: mentoring, grading, and scenario generation
- 8 checkpoint types including YARA rule challenges
- 5-dimension weighted scoring model
- Real-time trainer monitoring via WebSockets
- Fully self-hosted with MIT license
- Bring Your Own Key (BYOK) for Anthropic Claude API

---

2. System Overview

2.1 Platform Capabilities

Capability	Description
Scenario-Based Training	13 built-in scenarios across beginner, intermediate, and advanced levels
Investigation Workspace	10 log types, evidence collection, timeline building, process mapping
8 Checkpoint Types	True/False, Multiple Choice, Severity, Actions, Short Answer, Evidence Selection, Incident Report, YARA Rules
AI SOC Mentor	Context-aware Socratic guidance that never reveals answers
AI Scoring	AI-assisted grading for short answers, incident reports, and YARA rules
AI Scenario Generation	Generate complete scenarios from a text description

Real-Time Monitoring	Trainers observe trainee progress, send hints, and alerts via WebSockets
Reporting	PDF reports, CSV exports, leaderboards, and scenario analytics
RBAC	Three-role system with middleware-enforced authorization

2.2 User Roles

Role	Capabilities
Admin	Full system access: user management, scenario CRUD, audit logs, system settings
Trainer	Create/manage sessions, assign scenarios, monitor trainees in real-time, view reports
Trainee	Join sessions, investigate scenarios, use SOC Mentor, submit answers, review results

---

3. Architecture Overview

3.1 Monorepo Structure

```
soc-training-simulator/
├── client/          # Next.js 15 frontend (React 19)
├── server/          # Express 5 backend (Node.js)
├── shared/          # Shared TypeScript types
├── prisma/          # Database schema and migrations
├── e2e/             # Playwright E2E tests
├── docs/            # Architecture documentation
└── .github/         # CI/CD workflows
```

Workspace management: npm workspaces with four packages (`client` , `server` , `shared` , `prisma`).

3.2 Technology Stack Summary

Layer	Technology	Version
Frontend	Next.js (App Router)	15.x
UI Framework	React	19.x
State (Client)	Zustand + TanStack Query	5.x
UI Components	Radix UI + Tailwind CSS	—
Backend	Express	5.x
Real-Time	Socket.io	4.8
ORM	Prisma	6.2
Database	PostgreSQL	16
AI	Anthropic Claude (claude-sonnet-4-6)	—

Auth	JWT (jsonwebtoken) + bcryptjs	—
Validation	Zod	3.24
Testing	Vitest (unit) + Playwright (E2E)	—
CI/CD	GitHub Actions → Railway	—

3.3 Deployment Model

Self-hosted or cloud-deployed. Current production runs on **Railway** with automatic deploys on push to `master`.

---

4. System Context Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│                        EXTERNAL ACTORS                               │
│                                                                      │
│   ┌──────────┐     ┌──────────┐     ┌──────────┐                    │
│   │  Admin   │     │ Trainer  │     │ Trainee  │                    │
│   └──────────┘     └──────────┘     └──────────┘                    │
│        │              │              │                               │
└────────┼──────────────┼──────────────┼───────────────────────────────┘
         │              │              │
         │  HTTPS/WSS   │  HTTPS/WSS   │  HTTPS/WSS
         │              │              │
┌────────▼──────────────▼──────────────▼───────────────────────────────┐
│                                                                      │
│                  SOC TRAINING SIMULATOR                              │
│                                                                      │
│  ┌────────────────────┐     ┌────────────────────┐                  │
│  │   Client (Next.js) │◄────►│  Server (Express)  │                  │
│  │   Port 3000        │HTTP │  Port 3001         │                  │
│  │                    │ WS  │                    │                  │
│  └────────────────────┘     └────────────────────┘                  │
│                                    │      │                          │
└────────────────────────────────────┼──────┼──────────────────────────┘
                                     │      │
                              ┌──────▼┐  ┌─▼──────────────┐
                              │PostgreSQL │  │ Anthropic Claude │
                              │  (DB)     │  │    API           │
                              │           │  │  (Optional BYOK) │
                              └───────────┘  └──────────────────┘
```

---

5. Component Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                        CLIENT (Next.js 15)                           │
│                                                                      │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐               │
│  │  App Router  │  │   Zustand    │  │  TanStack    │               │
```

```
│  │  (Pages &     │  │  Auth Store  │  │   Query      │              │
│  │   Layouts)    │  │              │  │  (Server     │              │
│  │               │  │  (JWT token, │  │   State)     │              │
│  │  Route Groups:│  │   user,      │  │              │              │
│  │  (auth)       │  │   role)      │  │  staleTime:  │              │
│  │  (trainee)    │  │              │  │  30s         │              │
│  │  (trainer)    │  │  Persisted   │  │              │              │
│  │  (admin)      │  │  localStorage│  │              │              │
│  └───────────────┘  └──────────────┘  └──────────────┘              │
│         │                                                            │
│  ┌──────▼──────────────────────────────────────────────────────┐    │
│  │                    Investigation Workspace                   │  │
│  │  ┌────────────┐  ┌────────────┐  ┌────────────┐  ┌────────────┐  │  │
│  │  │ Briefing   │  │ Log Feed   │  │ Evidence   │  │ Checkpoint │  │  │
│  │  │ Panel      │  │ Viewer     │  │ & Timeline│  │ Modal      │  │  │
│  │  └────────────┘  └────────────┘  └────────────┘  └────────────┘  │  │
│  │  ┌────────────┐  ┌────────────┐                                 │  │
│  │  │ AI Mentor  │  │ Results    │                                 │  │
│  │  │ Panel      │  │ Screen     │                                 │  │
│  │  └────────────┘  └────────────┘                                 │  │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                      │
│  ┌──────────────┐  ┌──────────────┐                                  │
│  │  Axios       │  │  Socket.io   │                                  │
│  │  (REST API)  │  │  Client      │                                  │
│  │              │  │  /trainer    │                                  │
│  │  Auto-refresh│  │  /trainee   │                                  │
│  │  interceptor │  │              │                                  │
│  └──────────────┘  └──────────────┘                                  │
└──────────┬──────────────────┬──────────────────────────────────────┘
           │ HTTP             │ WebSocket
           │                  │
┌──────────▼──────────────────▼──────────────────────────────────────┐
│                        SERVER (Express 5)                            │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │                      Middleware Stack                        │    │
│  │  ┌────────┐ ┌──────┐ ┌────────┐ ┌──────┐ ┌──────┐ ┌──────┐ │    │
│  │  │Helmet  │ │ CORS │ │Global  │ │ CSRF │ │ Auth │ │ RBAC │ │    │
│  │  │(CSP,   │ │      │ │Rate    │ │Double│ │(JWT  │ │(Role │ │    │
│  │  │ HSTS,  │ │      │ │Limit   │ │Submit│ │Cookie│ │Check)│ │    │
│  │  │ Report)│ │      │ │200/min │ │Cookie│ │+Hdr) │ │      │ │    │
│  │  └────────┘ └──────┘ └────────┘ └──────┘ └──────┘ └──────┘ │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │                        REST API Routes                       │    │
│  │  /auth  /users  /scenarios  /sessions  /attempts             │    │
│  │  /logs  /reports  /ai  /yara  /messages                      │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐    │
```

```
│  │                    Socket.io Namespaces                        │    │
│  │  ┌──────────────────────┐  ┌──────────────────────┐          │    │
│  │  │ /trainer             │  │ /trainee             │          │    │
│  │  │ • join-session       │  │ • join-attempt       │          │    │
│  │  │ • send-hint          │  │ • progress-update    │          │    │
│  │  │ • send-session-alert │  │ • ai-assistant-msg   │          │    │
│  │  │ • pause/resume       │  │ • join-session       │          │    │
│  │  └──────────────────────┘  └──────────────────────┘          │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │                        Services                              │    │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐     │    │
│  │  │ Auth     │  │ Scoring  │  │ AI       │  │ YARA     │     │    │
│  │  │ Service  │  │ Service  │  │ Service  │  │ Service  │     │    │
│  │  └──────────┘  └──────────┘  └──────────┘  └──────────┘     │    │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐     │    │
│  │  │ PDF      │  │ Filter   │  │ Filter   │  │ Sanitize │     │    │
│  │  │ Report   │  │ AI Resp  │  │ AI Input │  │ Prompt   │     │    │
│  │  └──────────┘  └──────────┘  └──────────┘  └──────────┘     │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                      │
│  ┌──────────────┐                                                    │
│  │  Prisma ORM  │                                                    │
│  └──────────────┘                                                    │
└──────────┬───────────────────────────────────────────────────────────┘
           │ TCP/5432
           │
┌──────────▼──────────┐        ┌────────────────────────┐
│   PostgreSQL 16     │        │   Anthropic Claude API  │
│                     │        │                         │
│   16 models         │        │   Model: claude-sonnet-4-6│
│   7 enums           │        │   BYOK (optional)       │
│   42+ indexes       │        │                         │
└─────────────────────┘        └────────────────────────┘
```

---

6. Data Flow Overview

6.1 Authentication Flow

```
┌──────┐          ┌──────────┐          ┌──────────┐          ┌──────────┐
│Client│          │  Server  │          │ Auth     │          │ Database │
└──┬───┘          └────┬─────┘          │ Service  │          └────┬─────┘
   │                   │                └────┬─────┘               │
   │  POST /auth/login │                     │                     │
   │  {email,password} │                     │                     │
   │──────────────────►│                     │                     │
   │                   │  validate(email,pw) │                     │
   │                   │────────────────────►│                     │
   │                   │                     │  findUser(email)    │
   │                   │                     │────────────────────►│
```

```
   │                  │                    │◄──────────────────┤
   │                  │                    │  bcrypt.compare()  │
   │                  │                    │  sign JWT (4h)     │
   │                  │                    │  create RefreshToken│
   │                  │                    │──────────────────►│
   │                  │◄──────────────────┤                    │
   │  200 {token,user}│                    │                    │
   │  Set-Cookie:     │                    │                    │
   │  accessToken     │                    │                    │
   │  (httpOnly,4h)   │                    │                    │
   │  refreshToken    │                    │                    │
   │  (httpOnly,7d)   │                    │                    │
   │  csrf (readable) │                    │                    │
   │◄────────────────┤                    │                    │
   │                  │                    │                    │
   │  Store JWT in    │                    │                    │
   │  Zustand/memory  │                    │                    │
   │  (+ httpOnly     │                    │                    │
   │   cookie primary)│                    │                    │
   │                  │                    │                    │
   │  ── Later (401 on any request) ──     │                    │
   │                  │                    │                    │
   │  POST /auth/     │                    │                    │
   │  refresh (cookie)│                    │                    │
   │────────────────►│                    │                    │
   │                  │  rotateToken()     │                    │
   │                  │──────────────────►│                    │
   │                  │                    │  delete old token  │
   │                  │                    │  create new token  │
   │                  │                    │──────────────────►│
   │                  │◄──────────────────┤                    │
   │  200 {token}     │                    │                    │
   │  Set-Cookie: new │                    │                    │
   │◄────────────────┤                    │                    │
```

6.2 Training Session Lifecycle

```
┌─────────┐      ┌─────────┐      ┌─────────┐      ┌─────────┐
│ Trainer │      │ Server  │      │ Database│      │ Trainee │
└────┬────┘      └────┬────┘      └────┬────┘      └────┬────┘
     │                │                 │                 │
     │ 1. CREATE SESSION                │                 │
     │ POST /sessions │                 │                 │
     │ {name,scenario,│                 │                 │
     │  memberIds}    │                 │                 │
     │───────────────►│                 │                 │
     │                │  create session │                 │
     │                │────────────────►│                 │
     │◄───────────────┤                 │                 │
     │                │                 │                 │
     │ 2. START SESSION                 │                 │
```

```
    | PUT /sessions/ |                  |                  |
    | :id/status     |                  |                  |
    | {status:ACTIVE}|                  |                  |
    |————————————————►|                  |                  |
    |                |  update status   |                  |
    |                |——————————————————►|                  |
    |◄————————————————|                  |                  |
    |                |                  |                  |
    | 3. JOIN SESSION (Socket.io)       |  GET /sessions   |
    | emit:join-session                 |◄—————————————————|
    |————————————————►|                  |                  |
    |                |                  |                  |
    |                |                  |  4. START ATTEMPT |
    |                |                  |  POST /attempts/  |
    |                |                  |  start            |
    |                |◄——————————————————————————————————|
    |                |  create attempt  |                  |
    |                |——————————————————►|                  |
    |                |                  |——————————————————►|
    |                |                  |                  |
    |                |                  |  5. INVESTIGATE   |
    |                |                  |  GET /logs        |
    |                |                  |  POST /actions    |
    |                |                  |◄—————————————————|
    |                |                  |                  |
    |  progress-update (Socket.io)      |  emit:progress   |
    |◄——————————————————————————————————|——————————————————|
    |                |                  |                  |
    |                |                  |  6. ANSWER        |
    |                |                  |  POST /answers    |
    |                |◄——————————————————————————————————|
    |                |  grade + score   |                  |
    |                |——————————————————►|                  |
    |                |                  |——————————————————►|
    |                |                  |                  |
    |                |                  |  7. COMPLETE      |
    |                |                  |  POST /complete   |
    |                |◄——————————————————————————————————|
    |                |  final scoring   |                  |
    |                |——————————————————►|                  |
    |                |                  |——————————————————►|
    |                |                  |  (Results Screen) |
    |                |                  |                  |
    |  8. REVIEW REPORTS                |                  |
    |  GET /reports/ |                  |                  |
    |  session/:id   |                  |                  |
    |————————————————►|                  |                  |
    |◄————————————————|                  |                  |
```

6.3 AI Integration Flow

```
┌──────────────────────────────────────────────────────────────────────┐
│                        AI Integration Points                         │
│                                                                      │
│  ┌────────────────┐   ┌────────────────┐   ┌────────────────┐   │
│  │  SOC Mentor     │   │  AI Scoring     │   │ AI Scenario     │   │
│  │  (Real-time)    │   │  (Per-answer)   │   │ Generation      │   │
│  │                 │   │                 │   │                 │   │
│  │  Socket.io      │   │  REST API       │   │  REST API (SSE) │   │
│  │  /trainee ns    │   │  POST /answers  │   │  POST /ai/      │   │
│  │                 │   │                 │   │  generate       │   │
│  │  Limits:        │   │  Types:         │   │                 │   │
│  │  20/attempt     │   │  SHORT_ANSWER   │   │  Limits:        │   │
│  │  30/day/user    │   │  INCIDENT_REPORT │   │  5/day/user     │   │
│  │                 │   │  YARA_RULE      │   │                 │   │
│  │  4-layer output │   │  + feedback for │   │  Generates:     │   │
│  │  filter applied │   │    all 8 types  │   │  stages, logs,  │   │
│  │                 │   │                 │   │  checkpoints,   │   │
│  │  Socratic method │   │  Fallback to    │   │  hints          │   │
│  │  Never reveals  │   │  deterministic  │   │                 │   │
│  │  answers        │   │  if AI unavail. │   │  max_tokens:    │   │
│  │                 │   │                 │   │  8192           │   │
│  └────────────────┘   └────────────────┘   └────────────────┘   │
│                                                                      │
│  All AI features disabled gracefully when ANTHROPIC_API_KEY not set  │
└──────────────────────────────────────────────────────────────────────┘
```

---

7. Security Architecture

7.1 Authentication

Mechanism	Implementation
Password Hashing	bcryptjs with 12 salt rounds
Access Token	JWT HS256, 4-hour expiry, httpOnly cookie + Bearer header fallback
Refresh Token	JWT HS256, 7-day expiry, httpOnly secure cookie
CSRF Protection	Double-submit cookie pattern (non-httpOnly `csrf` cookie validated against `X-CSRF-Token` header)
Token Rotation	Old refresh token deleted on each refresh
Session Revocation	Logout deletes all user's refresh tokens + clears cookies
Password Change	Invalidates all active sessions (logoutAll)
Role Change Revocation	All refresh tokens deleted when user's role is modified
Account Lockout	Progressive lockout after 5 failed login attempts (15-min exponential backoff)

Default Credential Guard	Blocks login with demo credentials in production (`mustChangePassword` flag)

7.2 Authorization

Layer	Mechanism
Route Level	`requireRole('ADMIN', 'TRAINER')` middleware
Resource Level	Ownership checks (trainer owns session, trainee owns attempt)
Data Level	Sensitive fields stripped for TRAINEE role (correctAnswer, explanation, isEvidence)
Socket Level	Per-namespace role checks + per-event ownership validation + session membership checks
Token Revocation	Refresh tokens invalidated on role change or account deactivation

7.3 Rate Limiting

Target	Window	Limit	Storage
Global (all routes)	**60 sec**	**200 requests**	**In-memory**
Auth endpoints	15 min	15 requests	In-memory
Log endpoints	60 sec	100 requests	In-memory
YARA testing	60 sec	10 requests	In-memory
Actions tracking	**60 sec**	**60 requests**	**In-memory**
Socket events	10 sec	30 events	In-memory
AI Mentor (attempt)	Lifetime	20 messages	Database
AI Mentor (daily)	Calendar day	30 messages	Database
AI Scenario Gen	Calendar day	5 generations	Database
Socket connections	**Per-user**	**3 concurrent**	**In-memory**
YARA concurrency	**Server-wide**	**3 simultaneous**	**Semaphore**

7.4 AI Security (5-Layer Defense)

Input Filtering (`server/src/utils/filterAiInput.ts`):

- Detects and rejects ~30 jailbreak patterns before forwarding to AI (role overrides, system prompt extraction, DAN, bypass attempts)
- Blocked messages logged to audit trail with `AI_JAILBREAK_BLOCKED` action

Prompt Sanitization (`server/src/utils/sanitizePrompt.ts`):

- Strips prompt injection patterns from scenario content before AI system prompt injection

- Scans scenario briefing, stage titles, and descriptions for ~30 injection patterns
- Flags suspicious content during scenario creation with `contentWarnings`

Output Filtering (`server/src/utils/filterAiResponse.ts`) — 4-layer server-side filter:

1. **Phrase Detection** — Blocks answer-giving patterns ("the correct answer is", "you should select", etc.)
2. **Exact Answer Match** — Scans for checkpoint correctAnswer strings in response
3. **Explanation Overlap** — Flags if >60% of explanation words appear in response
4. **JSON Leak Detection** — Catches structured data containing correctAnswer/isEvidence fields

When triggered, the response is replaced with a Socratic redirect and an `AI_OUTPUT_FILTERED` audit log is created.

AI Conversation Review (`/ai-review` page):

- Trainer-facing dashboard for reviewing all AI conversations per session
- Anomaly flags (jailbreak blocked, output filtered) visible per trainee
- Full conversation transcript viewable with anomaly indicators

Cost Tracking:

- Estimated token usage and cost logged per AI call
- Checkpoint answers excluded from AI context to reduce unnecessary exposure

7.5 Additional Security

- **Helmet** security headers (CSP, HSTS, X-Frame-Options, X-Content-Type-Options)
- **CSP Violation Reporting** — `/api/csp-report` endpoint logs Content-Security-Policy violations (enabled via `CSP_REPORT_URI` env var)
- **CORS** with explicit origin whitelist and credentials support
- **Zod** schema validation on all API request bodies (including chat messages)
- **YARA sandboxing** — include/import directives stripped, 10s execution timeout, temp directory isolation, semaphore-based concurrency limit (max 3 simultaneous)
- **Audit logging** — all write operations + attempt start/complete + YARA executions + AI filter triggers + hints sent, with user ID, IP, action, and sanitized details
- **Sensitive field redaction** — passwords, tokens, secrets auto-redacted in audit logs
- **Prisma error sanitization** — database errors mapped to generic messages; model/field names never leaked to clients
- **WebSocket re-authentication** — periodic JWT verification (every 5 minutes) on active connections; disconnects expired sessions

---

8. Deployment Architecture

```
┌──────────────────┐      ┌──────────────────────────────────────────┐
│   Developer      │      │           GitHub                         │
│                  │      │                                          │
│  git push master─┼──────►│  ┌──────────────────────────────┐    │
│                  │      │  │        GitHub Actions            │    │
│                  │      │  │                                  │    │
│                  │      │  │  ci.yml:                         │    │
│                  │      │  │  ├── unit-test (Vitest)          │    │
│                  │      │  │  └── typecheck (tsc --noEmit)    │    │
│                  │      │  │                                  │    │
│                  │      │  │  deploy.yml:                     │    │
│                  │      │  │  ├── deploy-server ──────────┐   │    │
```

```
│                  │      │  │  └── deploy-client ────┐    │    │    │
│                  │      │  └────────────────────────┼────┼────┘    │
│                  │      └───────────────────────────┼────┼─────────┘
└──────────────────┘                                  │    │
                                                      │    │
                                                      │    │
                          ┌───────────────────────────▼────▼──────────┐
                          │              Railway                       │
                          │                                            │
                          │  ┌────────────────┐  ┌────────────────┐   │
                          │  │ Client Service  │  │ Server Service  │ │
                          │  │ (Next.js)       │  │ (Express)       │ │
                          │  │ standalone mode │  │ node dist/      │ │
                          │  │                 │  │ index.js        │ │
                          │  └────────┬────────┘  └────────┬────────┘ │
                          │           │                    │          │
                          │           │   ┌────────────────┘          │
                          │           │   │                           │
                          │  ┌────────▼───▼──────┐                    │
                          │  │ PostgreSQL 16     │                    │
                          │  │ (Railway          │                    │
                          │  │  managed)         │                    │
                          │  └────────────────────┘                    │
                          └────────────────────────────────────────────┘
```

Deployment Pipeline:

1. Developer pushes to `master`
2. **ci.yml** runs unit tests (Vitest) and type checks (tsc) in parallel
3. **deploy.yml** deploys server and client services to Railway in parallel
4. Railway builds and restarts services (~2 minutes total)

---

9. Technology Stack

Category	Technology	Version	Purpose
Frontend	Next.js	15.x	React framework with App Router (SSR/CSR)
	React	19.x	UI component library
	Zustand	5.x	Client-side auth state management
	TanStack Query	5.x	Server state caching and synchronization
	Tailwind CSS	3.4	Utility-first CSS framework
	Radix UI	—	Accessible headless UI components
	Axios	1.7	HTTP client with interceptors
	Socket.io Client	4.8	WebSocket client for real-time features
	Recharts	2.15	Charts and data visualization
	Lucide React	0.468	Icon library

	next-themes	0.4	Dark/light mode theme management
Backend	Express	5.x	HTTP server framework
	Socket.io	4.8	WebSocket server for real-time communication
	Prisma	6.2	Type-safe ORM with migration support
	Zod	3.24	Runtime schema validation
	jsonwebtoken	9.0	JWT token generation and verification
	bcryptjs	2.4	Password hashing
	Helmet	8.0	Security headers middleware
	express-rate-limit	7.5	API rate limiting
	Winston	3.17	Structured logging
	PDFKit	0.16	PDF report generation
	csv-stringify	6.5	CSV export generation
	cookie-parser	1.4	HTTP cookie parsing
AI	Anthropic Claude SDK	0.78	AI API client (BYOK)
Database	PostgreSQL	16	Relational database
Testing	Vitest	4.0	Unit testing framework (40 tests)
	Playwright	1.58	E2E browser testing (66 tests)
DevOps	GitHub Actions	—	CI/CD pipelines
	Railway	—	Cloud hosting platform
Language	TypeScript	5.7	Type-safe JavaScript (strict mode)

---

10. Key Design Decisions

Decision	Rationale
Express 5 + Next.js 15	Separate server allows Socket.io integration, independent scaling, and clear API boundaries
Socket.io over plain WebSockets	Built-in rooms, namespaces, reconnection, and fallback transport for real-time monitoring
Anthropic Claude (BYOK)	Socratic mentoring requires strong instruction-following; BYOK keeps platform free to host
Prisma ORM	Type-safe database access, auto-generated client, migration tooling, and PostgreSQL optimization

JWT dual-token httpOnly auth	Both access (4h) and refresh (7d) tokens in httpOnly cookies + CSRF double-submit pattern
Zustand over Redux	Minimal boilerplate for simple auth state; TanStack Query handles server state
Monorepo with npm workspaces	Shared types between client/server; single repo for simpler CI/CD and contribution
MIT License	Maximizes adoption in the open-source SOC training space; CLA preserves commercial optionality
5-dimension scoring	Measures distinct analyst competencies rather than a single score, enabling targeted improvement
AI 5-layer security	Input filtering + prompt sanitization + 4-layer output filter + conversation review + cost tracking

---

11. Non-Functional Requirements

Requirement	Target
Scalability	Supports concurrent training sessions; stateless server allows horizontal scaling
Performance	Log pagination (50/page), TanStack Query caching (30s stale), Socket.io rooms
Availability	Railway auto-restart; graceful error handling with AppError class
Security	JWT httpOnly + CSRF + RBAC + rate limiting (global + per-route) + CSP + Zod + AI 5-layer defense + account lockout + Prisma error sanitization
Extensibility	Modular service architecture; scenario JSON format for community contributions
Accessibility	Radix UI primitives provide ARIA attributes; WCAG audit deferred
Observability	Winston structured logging; audit log table; AI conversation storage

---

12. External Integrations

12.1 Anthropic Claude API

Feature	Model	Max Tokens	Rate Limit
SOC Mentor	claude-sonnet-4-6	500	20/attempt, 30/day
Short Answer Grading	claude-sonnet-4-6	512	Per-answer
Report Grading	claude-sonnet-4-6	512	Per-answer
YARA Rule Feedback	claude-sonnet-4-6	512	Per-answer
Checkpoint Feedback	claude-sonnet-4-6	256	Per-answer
Scenario Generation	claude-sonnet-4-6	8192	5/day/user

Graceful Degradation: All AI features disabled when API key not configured. Scoring falls back to deterministic keyword matching.

12.2 YARA Engine

YARA rules are compiled and tested server-side with sandboxing:

- Include/import directives stripped
- Execution timeout: 10 seconds
- Isolated temp directory per test
- Max 10 samples, 1MB each

---

13. Future Considerations

Items discussed by the advisory board but not yet implemented:

Item	Phase	Description
Adaptive AI difficulty	Phase 3	AI observes behavior and adjusts guidance proactively
SSO / SAML support	Phase 3	Enterprise authentication integration
Scenario marketplace	Phase 3	Community-contributed scenarios with network effects
Sigma/KQL/SPL challenge types	Phase 3	Additional detection engineering exercises
Gamification	Phase 3	Badges, leaderboards, streaks, team competitions
Collaborative investigation mode	Phase 3	Multi-trainee real-time investigation
Threat intel feed integration	Future	Auto-generate scenarios from real-world incidents
Integration tests	Next	Tests hitting actual API routes with a test database
Accessibility audit	Next	WCAG compliance check for the investigation workspace
Monitoring & observability	Next	Structured logging dashboards, health checks, error tracking
Remove `unsafe-inline` from CSP	Next	Use CSS nonces/hashes instead of blanket unsafe-inline
Semantic AI response analysis	Future	Embedding-based similarity check between AI responses and answers

---

Appendix: Glossary

Term	Definition

Attempt	A single trainee's investigation run within a session
Checkpoint	A question or task the trainee must complete during investigation
Evidence	Logs marked by the trainee as relevant to the investigation
MITRE ATT&CK	Framework of adversary tactics and techniques mapped to scenarios
Scenario	A complete training exercise with stages, logs, checkpoints, and hints
Session	A trainer-created training event that groups trainees around a scenario
SOC Mentor	AI-powered assistant that uses Socratic questioning to guide trainees
Stage	A phase within a scenario that unlocks progressively during investigation
YARA Rule	Pattern-matching rules used to identify malware; a checkpoint type
BYOK	Bring Your Own Key — users provide their own Anthropic API key